

AI Sustainability Copilot (Agentic AI + GenAI + Carbon Data)

First of all, let's know

What is Carbon Footprint?

A **carbon footprint** is the total amount of harmful greenhouse gases — mainly carbon dioxide (CO₂) — that are released into the atmosphere because of our everyday activities.

It includes things like:

- How we travel (car, bike, bus, flight)
- How much electricity we use
- What we eat and buy
- Even how websites or apps we use are hosted on servers

In short:

Carbon footprint = the pollution we cause through our lifestyle, choices, and energy usage.

Problem Statement (Simple and Clear)

Most people do not know **how their daily actions affect the environment**.

Whatever tools exist today:

- Only show numbers (like “You produced 200 kg CO₂”)
- Do not explain what that number means
- Do not tell what changes will actually help the person
- Do not use real-time information (like weather, energy grid, public transport)
- Are not interactive or personalized

So the **main problem** we are solving:

There is no smart and user-friendly system that can calculate a person's carbon footprint and also give personalized, real-time, and easy-to-understand suggestions to reduce it.

- **Solution:**
 - Collect data (electricity, travel, food habits).
 - ML → calculate footprint.
 - **GenAI Agent** → analyzes data + suggests **actionable eco-friendly alternatives** (“switch to LED bulbs, you'll save X CO₂ per year”).
 - **MCP** → connect to APIs (weather API, electricity rates API) → dynamic recommendations.
- **UI:** Chat interface (React/Next.js) → user chats with “Green AI Assistant.”

What It Will Do (Workflow)

1. **User Input** → User enters lifestyle/activity data (travel, electricity usage, diet, shopping habits).
2. **ML Model** → Calculates carbon footprint (CO₂ emissions estimate).
3. **GenAI Agent** → Analyzes results, explains them in natural language, and suggests personalized eco-friendly alternatives.
4. **MCP Connectors** → Pull in external data:
 - Weather API → Suggest best days for cycling instead of driving.
 - Electricity rates API → Show money saved by reducing consumption.
 - Transport API → Suggest public transport alternatives.
5. **UI** → A chat-like dashboard where the user interacts with the Copilot.

Models We'll Use

1. **Carbon Footprint Estimator (ML model)**
 - Dataset:
 - Carbon Footprint Datasets on Kaggle.
 - UCI ML datasets on energy consumption, transport CO₂, etc.
 - Approach:
 - Regression model like (XGBoost, RandomForest, or Neural Net) will be used.
 - Inputs: distance traveled, electricity usage, food type, shopping habits.
 - Output: Estimated CO₂ emissions (kg/year).
2. **GenAI Layer (LLM)**
 - Model: GPT-4, LLaMA-3, or Mixtral (via OpenAI API or Hugging Face).
 - Purpose:
 - Take ML output and explain it in **human-friendly way**.
 - Generate **personalized recommendations**:
"You can save ~120kg CO₂/year by switching from car to bus for your daily commute."
3. **Agentic AI (Orchestration)**
 - Tool: LangGraph, CrewAI, or custom Python agents.
 - Agents:

- **Calculator Agent:** Runs your ML model to compute footprint.
- **Research Agent:** Uses MCP to fetch live data (weather, electricity rates, transport).
- **Advisor Agent:** Uses LLM to give personalized suggestions.

How We'll Implement It

Step 1: Collect & Train

- Preprocess dataset (Python + pandas + scikit-learn).
- Train regression/classification model → Export (joblib/pickle).

Step 2: Backend

- FastAPI / Spring Boot API with endpoints:
 - /predict → takes user lifestyle data, returns carbon footprint score.
 - /advice → calls LLM + Agent layer to generate suggestions.

Step 3: GenAI Agent Layer

- Use LangGraph (Python) to chain:
 - User Input → ML Prediction → MCP (API fetch) → LLM Advice.
- Example flow:
 - ML says: 1,200 kg CO₂/year.
 - MCP fetches electricity rates + weather.
 - LLM responds:

"Your footprint is 20% higher than average. Switching to solar + biking twice a week could save you ~300kg CO₂ and ₹5,000 annually."

Step 4: MCP Integration

- Build connectors to:
 - Weather API (OpenWeather).
 - Energy data API (electricity rates, renewable stats).
 - Transport data API (e.g., Google Maps Directions API).

Step 5: UI

- React/Next.js + Tailwind → Chat interface.
- Dashboard with:
 - Carbon footprint graph.
 - Recommendations panel.

- “Green Score” gamification.

Step 6: Deployment

- Containerize (Docker).
- Deploy API + frontend on Render/AWS.
- CI/CD with GitHub Actions.

Key papers & resources for the same Idea

1. **“Carbon Emission Quantification of Machine Learning: A Review” — IEEE Transactions on Sustainable Computing (2025)**
A recent IEEE review paper summarising methods to estimate ML carbon emissions and best practices for reporting training/inference footprints. Important for the “how to measure your model’s own emissions” section. [ResearchGate](#)
2. **Articles on AI & sustainability (IEEE Spectrum, Research Group reports)** — e.g., “Measuring AI’s Carbon Footprint” (IEEE Spectrum) and recent work on sustainable LLM agents. These are good background and to cite concerns about LLM deployment carbon cost.
[IEEE Spectrum](#)

Is there already something like AI Sustainability Copilot in IEEE / literature?

- **There is published work on carbon-emission prediction** (national/industrial/urban levels) and on **measuring the carbon footprint of ML models** (JMLR, IEEE reviews, arXiv). We found multiple review and applied papers covering ML models for emissions prediction and tools for measuring ML energy.
 - **However, the exact product we propose — an end-user “Sustainability Copilot” that combines a small/local carbon-estimation ML model + an LLM-based GenAI advisor + agentic MCP connectors (weather, rates, transport) — appears to be relatively unintroduced in the academic literature.** Existing work tends to focus on: (a) estimating emissions at large scale, (b) measuring ML training footprint, or (c) separate sustainability tools from big vendors. But no peer-reviewed papers combine a user-level estimator + agentic GenAI assistant + MCP orchestration as a packaged system with user studies.
-